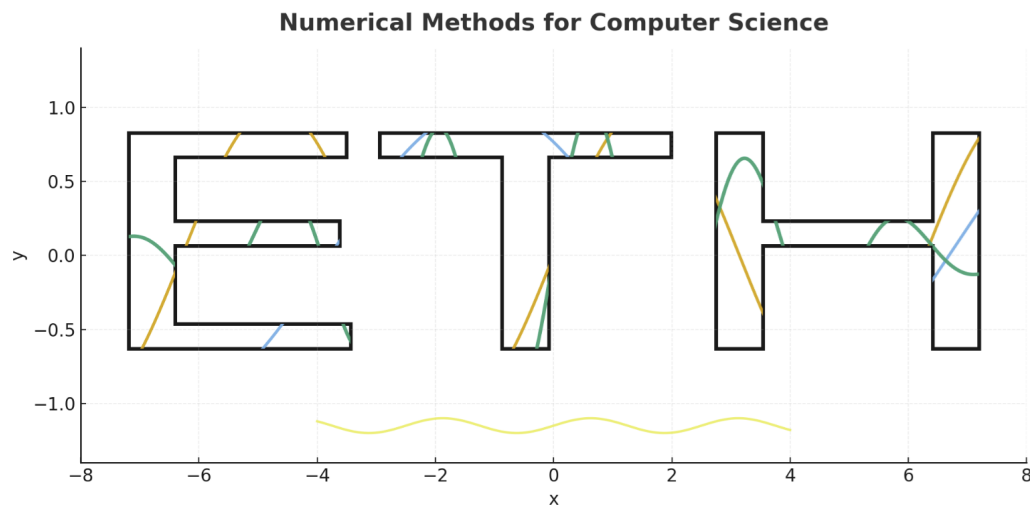




Advanced Methods for Nonlinear Systems — Week #9

ETH Zurich

DAVID Mihnea-Ştefan

mihdavid@ethz.ch

This week we advance from scalar root-finding toward multidimensional systems.

We extend Newton's ideas to vector functions, analyze Jacobians and convergence in $\mathbb{R}^n$, and connect root-finding with optimization, exploring how gradient and Hessian information guide us to minima and equilibria.

Contents

1	Recap	1
1.1	What Defines a Nonlinear Equation/System	1
1.2	Iterative Methods	2
1.3	Fixed Points and Their Significance	2
1.4	Convergence of Iterative Methods	5
1.5	Theoretical Guarantees of Convergence	8
1.6	Newton's Method (Scalar Case)	9
1.7	Multi-Point and Higher-Order Methods	11
2	Multidimensional Newton's Method	14
2.1	Idea and Derivation	14
2.2	Implementation Viewpoint	14
2.3	Connection to Fixed-Point Form and Local Behavior	15
3	Damped/Line Search Newton Method	17
3.1	Motivation (1D Intuition)	17
3.2	(Optional) Choosing the Damping Parameter	18
3.3	Summary	19
4	Quasi-Newton (Broyden) Method	20
4.1	From Secant to Broyden's Update	20

TA Award 2025

A note from your TA



Hello everyone! As you've probably already seen, VIS is once again organizing the **TA Award**, a prize given by students to the TA who resonated the most with them. If you feel that this might be me, and that I've managed, even a little, to make the NumCS course more digestible or interesting, I would be truly grateful if you considered voting for me.

On your side, it only takes a minute to open the link and cast your vote, but for me it would be a real confirmation that what I do actually has an impact. Knowing that my work has helped you, even in a small way, would mean a lot.

Thank you very much for your time and support!

1 Recap

Iterative methods form the backbone of nonlinear numerical analysis. Unlike linear solvers, which can often produce an exact answer in a finite number of steps, nonlinear equations usually demand *progressive approximation*.

1.1 What Defines a Nonlinear Equation/System

A **nonlinear equation** is any equation where the unknown appears inside a nonlinear operation:

$$F(x) = 0, \quad \text{with } F: \mathbb{R}^n \rightarrow \mathbb{R}^n$$

and at least one component of F is not a linear combination of the variables.

Typical nonlinearities include:

- products or powers of unknowns, e.g. x_1x_2 , x^2 , $\sin(x)$, e^x ;
- coefficients that depend on the solution itself;
- implicit or transcendental relationships.

Example (Scalar vs. Vector Nonlinearity).

Scalar case:

$$f(x) = e^{-x} - x = 0.$$

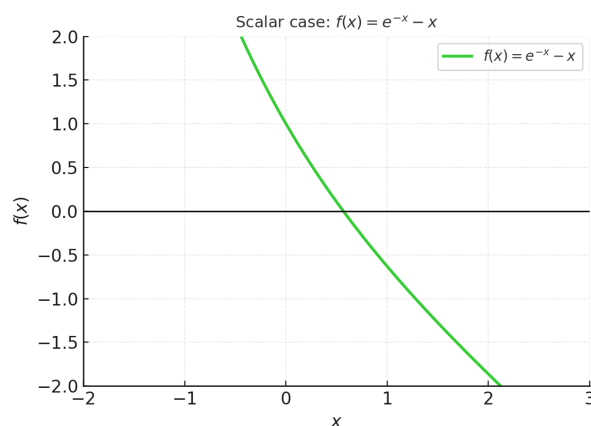
The function mixes exponential and linear terms, no algebraic solution exists.

Vector case:

$$\mathbf{F}(\mathbf{x}) = \begin{bmatrix} x_1^2 + x_2^2 - 4 \\ e^{x_1} + x_2 - 1 \end{bmatrix} = \mathbf{0}.$$

This couples two nonlinear equations; the solution must be found numerically.

Nonlinear Behavior: Scalar (2D) vs. Vector (3D) Examples



Vector case: $z = x_1^2 + x_2^2 - 4$ and $z = e^{x_1} + x_2 - 1$

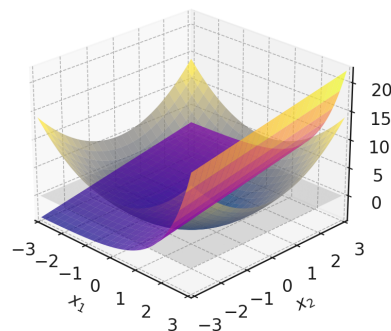


Figure:

Nonlinear scalar (left) and vector (right) examples. Both involve nonlinear dependencies, yet the vector case couples variables, leading to richer intersection geometry in $\mathbb{R}^3$.

1.2 Iterative Methods

An **iterative method** builds a sequence of approximations:

$$x^{(0)}, x^{(1)}, x^{(2)}, \dots \longrightarrow x^*,$$

where each iterate is computed from previous ones by an *iteration function*:

$$x^{(k+1)} = \Phi(x^{(k)}),$$

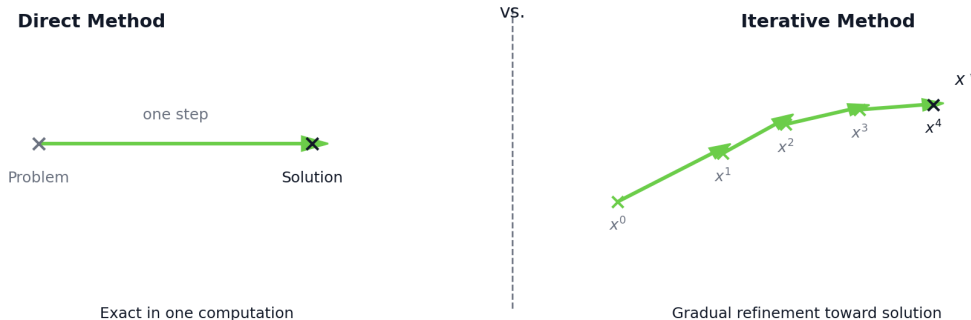
or more generally, for m -point methods:

$$x^{(k+1)} = \Phi_F(x^{(k)}, x^{(k-1)}, \dots, x^{(k-m+1)}).$$

The first m elements $x^{(0)}, x^{(1)}, \dots, x^{(m-1)}$ are called **initial guesses** or **guess points**, chosen by the user at the start of the algorithm. They serve as the entry point for the iteration process and can be selected based on prior knowledge, physical intuition, or even at random.

Note

The function Φ_F defines the entire algorithm. It maps current estimates into new ones, and its fixed points correspond to the solutions of the original system $F(x) = 0$. The performance and stability of the method depend not only on Φ_F but also on how well the initial m guesses approximate the true root.



Direct vs. Iterative solution. A direct method computes the answer in one step, while an iterative method refines successive approximations until reaching the target solution x^* .

1.3 Fixed Points and Their Significance

A **fixed point** of a mapping $\Phi : D \subset \mathbb{R}^n \rightarrow \mathbb{R}^n$ is a vector x^* such that

$$\Phi(x^*) = x^*.$$

Intuitively, applying Φ to x^* leaves it unchanged: the iteration “stops moving.” In numerical methods, this property encodes the very definition of having *found the solution* (because we

can't reduce the error anymore).

Suppose our original nonlinear problem is

$$F(x) = 0.$$

If we can rewrite it in an equivalent form

$$x = \Phi(x),$$

then the fixed point x^* of Φ automatically satisfies $F(x^*) = 0$. This connection is the cornerstone of all iterative methods: finding a root of F is the same as finding a fixed point of Φ .

Note

A well-designed iteration must be *consistent*:

$$\Phi(x^*) = x^* \quad \Leftrightarrow \quad F(x^*) = 0.$$

Otherwise, the fixed point of Φ might not correspond to a true root of F . Consistency ensures that the algorithm's stationary points are the correct mathematical solutions.

Example: From root finding to fixed point. Consider the nonlinear equation

$$F(x) = \cos x - x = 0.$$

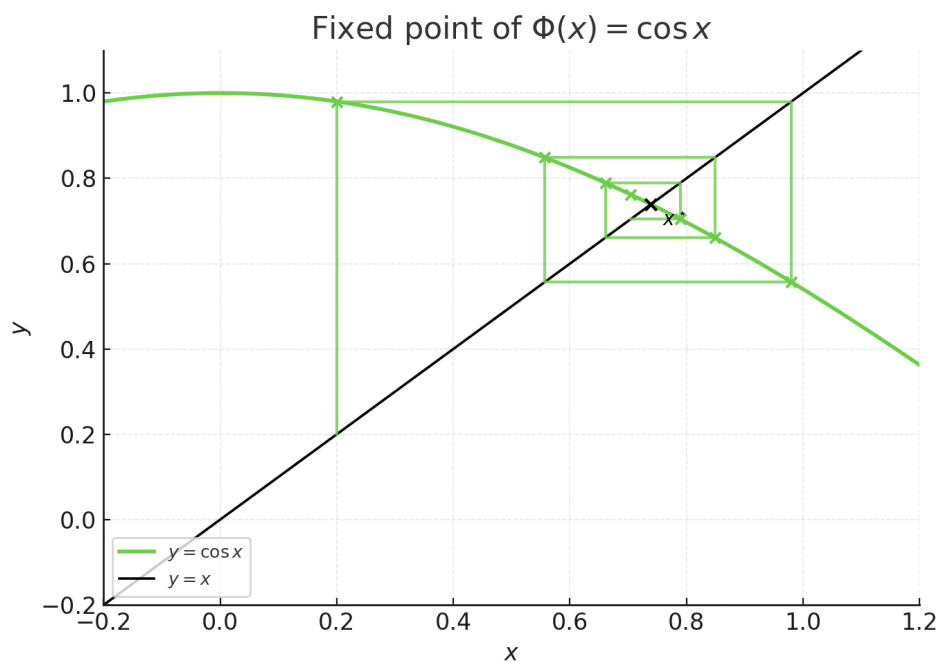


Figure: The root of $F(x) = \cos x - x$ is the intersection between $y = \cos x$ and $y = x$. The same point x^* is the fixed point of the mapping $\Phi(x) = \cos x$.

Geometric and conceptual meaning. Visually, a fixed point is the intersection of the curve $y = \Phi(x)$ with the diagonal $y = x$. Each iteration moves from x_k vertically to $y = \Phi(x)$ (evaluation step), then horizontally back to $y = x$ (projection step). Repeating this motion generates the familiar *cobweb diagram*, whose pattern directly reveals the behavior of the iteration.

The key factor governing convergence is the slope of Φ at the fixed point:

$$|\Phi'(x^*)| < 1 \quad \Rightarrow \quad \text{local convergence.}$$

Note

This criterion comes from the first-order local error model:

$$e_{k+1} = \Phi'(x^*)e_k + \mathcal{O}(e_k^2),$$

so if $|\Phi'(x^*)| < 1$, the error decreases approximately by a factor $|\Phi'(x^*)|$ each step (so at least linear).

Intuitively, this means that Φ is “flat enough” near the intersection, nearby points are pulled inward rather than pushed away.

- If $|\Phi'(x^*)| < 1$, the staircase contracts toward x^* .
- If $|\Phi'(x^*)| > 1$, the staircase expands and diverges.
- If $|\Phi'(x^*)| = 1$, we are in a neutral or oscillatory situation: convergence is not guaranteed and small perturbations can lead to cycles.

Fixed Point Geometry — Effect of the Derivative

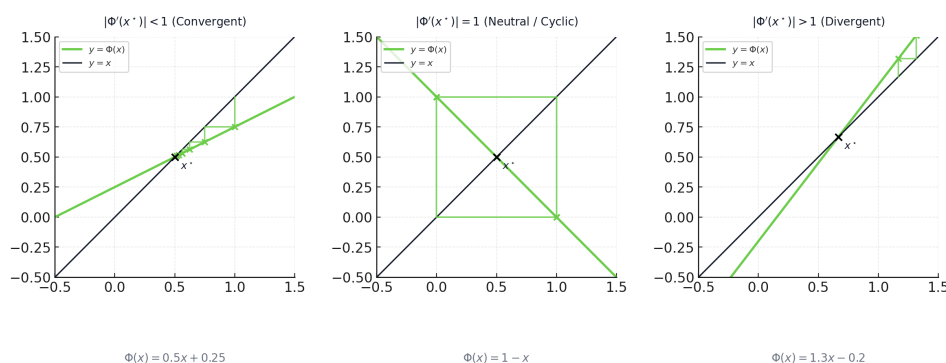


Figure: Fixed point as intersection of $y = x$ and $y = \Phi(x)$ (1D case). The slope $|\Phi'(x^*)|$ determines whether iterates spiral inward (stable) or outward (unstable).

Example: why the cosine map is safe. For $\Phi(x) = \cos x$, we have $\Phi'(x) = -\sin x$. Since $|\sin x| \leq 1$ for all real x , we immediately know that

$$|\Phi'(x)| = |\sin x| < 1$$

except at isolated points where $|\sin x| = 1$ (which do not correspond to the fixed point).

Note

This is a powerful idea: sometimes we can guarantee convergence *without computing the solution itself*, simply by bounding the derivative. The derivative acts as a “stability detector”, its magnitude tells us whether the iteration pulls errors inward or amplifies them.

1.4 Convergence of Iterative Methods

Once an iterative scheme has been defined, the next question is not *if* it converges, but *how fast*. Convergence analysis provides the mathematical tools to measure and compare the efficiency of different algorithms.

Definition of Convergence Let $\{x^{(k)}\}_{k=0}^{\infty}$ be the sequence generated by an iteration. We say that the method **converges** to x^* if

$$\lim_{k \rightarrow \infty} x^{(k)} = x^*.$$

The difference between the current iterate and the true solution is the **error**:

$$e^{(k)} = x^{(k)} - x^*.$$

Note

Convergence alone only guarantees that $x^{(k)}$ gets closer to x^* as k increases, but says nothing about the *speed* of that approach. Different methods can converge to the same solution with vastly different rates.

Types of Convergence To characterize how quickly the sequence $\{x^{(k)}\}$ approaches x^* , we compare successive errors. A method is said to have **order of convergence** $p > 0$ if

$$\lim_{k \rightarrow \infty} \frac{|e^{(k+1)}|}{|e^{(k)}|^p} = C,$$

for some constant $C > 0$, called the **asymptotic error constant**. Intuitively:

- $p = 1$ — **linear convergence**: error shrinks by a constant factor at each step.
- $p > 1$ — **superlinear convergence**: error decreases faster than linearly.
- $p = 2$ — **quadratic convergence**: the number of correct digits roughly doubles each iteration (we will see this in practice soon).

For example, if $|e^{(k+1)}| \approx 0.1 |e^{(k)}|$, then $p = 1$ and $C = 0.1$ (slow, steady progress). If instead $|e^{(k+1)}| \approx (|e^{(k)}|)^2$, then $p = 2$ and convergence accelerates dramatically near the root.

Example: Different Fixed-Point Forms, Different Behaviors Consider the nonlinear equation

$$F(x) = xe^x - 1 = 0,$$

whose true root is approximately $x^* \approx 0.567143$ (the so-called *Omega constant*).

We can rewrite $F(x) = 0$ in several equivalent fixed-point forms:

$$\Phi_1(x) = e^{-x}, \quad \Phi_2(x) = \frac{1+x}{1+e^x}, \quad \Phi_3(x) = x + 1 - xe^x.$$

Each formulation defines a different iteration scheme:

$$x_{k+1} = \Phi_i(x_k), \quad i = 1, 2, 3.$$

Experiment. Starting from the same initial guess $x^{(0)} = 0$, we compute the iterates for each variant up to five steps.

Iteration	$x^{(0)}$	$x^{(1)}$	$x^{(2)}$	$x^{(3)}$	$x^{(4)}$	$x^{(5)}$
$\Phi_1(x) = e^{-x}$	0.000	1.000	0.368	0.692	0.501	0.607
$\Phi_2(x) = \frac{1+x}{1+e^x}$	0.000	0.500	0.565	0.567	0.567	0.567
$\Phi_3(x) = x + 1 - xe^x$	0.000	1.000	0.000	1.000	0.000	1.000

Observations.

- Φ_1 : converges slowly but steadily. Indeed, $\Phi'_1(x^*) = -e^{-x^*} \approx -0.567$, so convergence is linear with ratio $\lambda \approx 0.57$.
- Φ_2 : converges very quickly. Here, $\Phi'_2(x^*) \approx 0.003$, indicating extremely fast linear (almost quadratic) convergence.
- Φ_3 : oscillates forever between 0 and 1 — divergence. Since $\Phi'_3(x^*) \approx 1.56 > 1$, the fixed point is unstable.

Note

This example illustrates that even though all three Φ_i are algebraically equivalent to the same equation $F(x) = 0$, only some of them lead to a convergent iteration. The derivative $|\Phi'(x^*)|$ fully determines the local behavior.

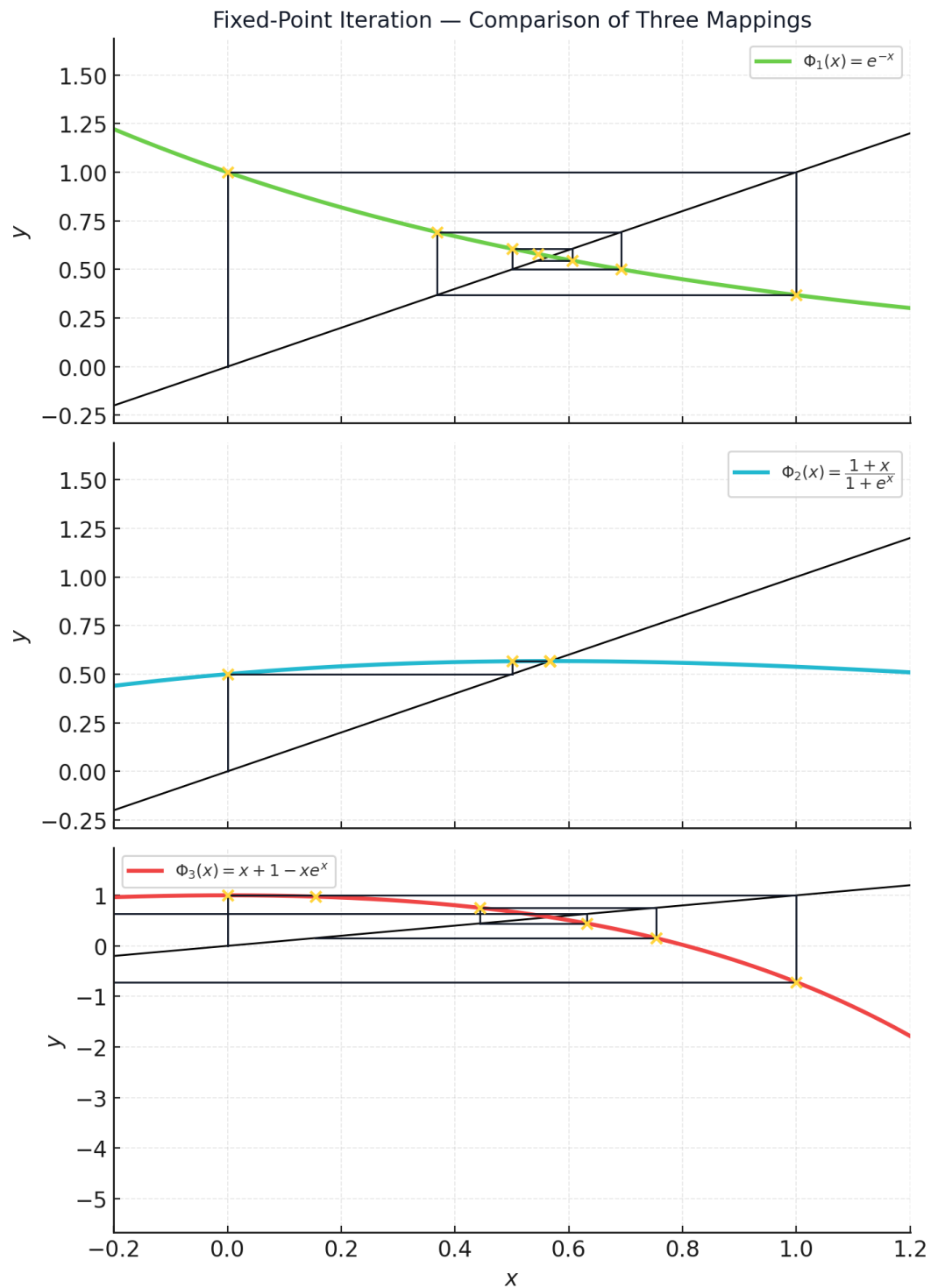


Figure: Three fixed-point functions for $F(x) = xe^x - 1$. Only Φ_1 and Φ_2 cross $y = x$ with slope magnitude < 1 .

1.5 Theoretical Guarantees of Convergence

So far, we have reasoned intuitively that if the slope of the iteration function $\Phi(x)$ near the fixed point is sufficiently small, then the sequence

$$x^{(k+1)} = \Phi(x^{(k)})$$

should converge. We now state the precise mathematical conditions that make this true.

Local Linear Convergence. Let $\Phi : U \subset \mathbb{R} \rightarrow \mathbb{R}$ be continuously differentiable, and suppose $\Phi(x^*) = x^*$. If the derivative at the fixed point satisfies

$$|\Phi'(x^*)| < 1,$$

then the iteration converges *locally and at least linearly* to x^* , that is,

$$\exists L \in (0, 1) : \quad |x^{(k+1)} - x^*| \leq L |x^{(k)} - x^*|, \quad \text{for all } k \text{ sufficiently large.}$$

This criterion follows from the first-order local error model:

$$e_{k+1} = \Phi'(x^*)e_k + \mathcal{O}(e_k^2),$$

so if $|\Phi'(x^*)| < 1$, the error decreases approximately by a factor $|\Phi'(x^*)|$ at each step.

Note

In higher dimensions, the condition becomes $\|D\Phi(x^*)\| < 1$, where $D\Phi(x^*)$ is the Jacobian matrix of Φ . It means that the iteration acts as a *local contraction* near the fixed point.

Interpretation. This result guarantees at least geometric (linear) convergence. However, it does not exclude faster convergence rates. To see why, expand Φ in a Taylor series near x^* :

$$\Phi(x) = \Phi(x^*) + \Phi'(x^*)(x - x^*) + \frac{1}{2}\Phi''(x^*)(x - x^*)^2 + \mathcal{O}((x - x^*)^3).$$

If $\Phi'(x^*)$ is small or even zero, the higher-order terms dominate, leading to faster-than-linear convergence.

Higher-Order Convergence. If Φ is $(m+1)$ -times continuously differentiable and

$$\Phi^{(l)}(x^*) = 0, \quad \text{for all } l = 1, \dots, m,$$

then the fixed-point iteration converges *locally* to x^* with order at least $m+1$:

$$|x^{(k+1)} - x^*| \leq C |x^{(k)} - x^*|^{m+1}.$$

Why “at least” linear? The term “at least” reflects that linear convergence is only the minimal rate guaranteed by the derivative test. If $\Phi'(x^*)$ happens to vanish, the iteration immediately becomes superlinear or quadratic.

Condition near x^*	Resulting convergence order
$ \Phi'(x^*) < 1$	Linear
$\Phi'(x^*) = 0, \Phi''(x^*) \neq 0$	Quadratic
$\Phi'(x^*) = \Phi''(x^*) = 0, \Phi'''(x^*) \neq 0$	Cubic

Note

This theoretical framework unifies all iterative root-finding methods. Newton’s and Secant methods can both be expressed as fixed-point iterations with specially designed $\Phi(x)$ that satisfy $\Phi'(x^*) = 0$, hence achieving superlinear or quadratic convergence.

1.6 Newton’s Method (Scalar Case)

Among all iterative schemes, **Newton’s method** plays a central role. It refines the general fixed-point concept into a powerful root-finding technique with *quadratic convergence* near the solution.

Idea and Derivation We seek a root of $f(x) = 0$. At iteration k , assume we have an approximation $x^{(k)}$. Using a first-order Taylor expansion of f around $x^{(k)}$:

$$f(x) \approx f(x^{(k)}) + f'(x^{(k)})(x - x^{(k)}).$$

Setting this linear model to zero gives an estimate of the root:

$$0 = f(x^{(k)}) + f'(x^{(k)})(x^{(k+1)} - x^{(k)}),$$

from which we obtain the classical Newton update:

$$x^{(k+1)} = x^{(k)} - \frac{f(x^{(k)})}{f'(x^{(k)})}.$$

Each step uses the tangent line of f at $x^{(k)}$ to approximate where it crosses the x -axis. Hence, Newton’s method can also be seen as a specific *fixed-point iteration* with

$$\Phi(x) = x - \frac{f(x)}{f'(x)}.$$

Geometric Interpretation Geometrically, at each step we draw the tangent to $y = f(x)$ at the current point and move to its intersection with the x -axis.

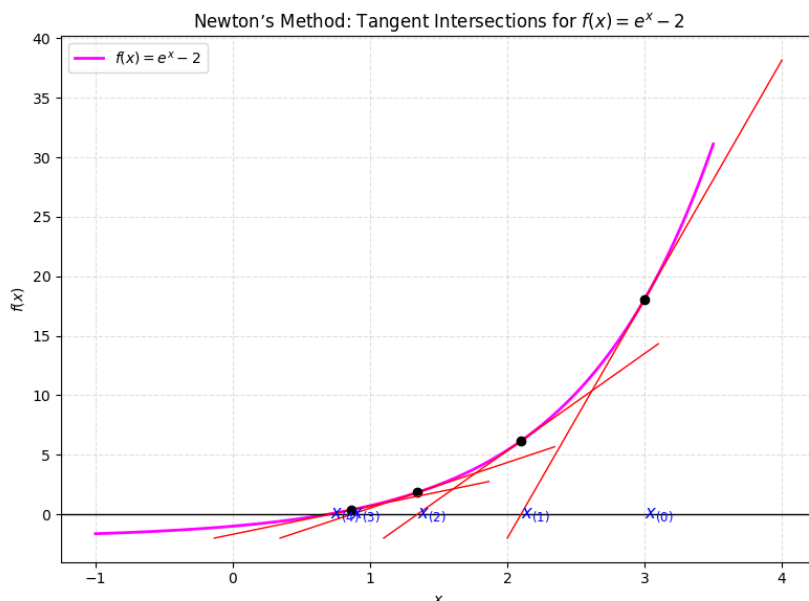


Figure: Newton's method as tangent intersection. The next iterate x_{k+1} is where the tangent at x_k crosses the axis.

Connection to Fixed-Point Iteration Formally, Newton's method is a particular case of the fixed-point scheme:

$$x_{k+1} = \Phi(x_k), \quad \Phi(x) = x - \frac{f(x)}{f'(x)}.$$

To find its derivative, we apply the quotient rule to the term $\frac{f(x)}{f'(x)}$:

$$\frac{d}{dx} \left(\frac{f(x)}{f'(x)} \right) = \frac{f'(x) f'(x) - f(x) f''(x)}{[f'(x)]^2} = 1 - \frac{f(x) f''(x)}{[f'(x)]^2}.$$

Substituting this back into $\Phi'(x) = 1 - \frac{d}{dx} \left(\frac{f(x)}{f'(x)} \right)$ gives

$$\boxed{\Phi'(x) = \frac{f(x) f''(x)}{[f'(x)]^2}}.$$

At the true root x^* , we have $f(x^*) = 0$, hence

$$\Phi'(x^*) = 0.$$

This automatically satisfies the convergence condition $|\Phi'(x^*)| < 1$, and in fact implies *quadratic convergence*.

Sensitivity to the Initial Guess Newton's method converges very rapidly *if* the starting point is close to the root. However, if x_0 is far away, the tangent may point in the wrong direction and jump to an entirely different region of the function.

$$x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)}.$$

Although Newton's method is locally powerful, it is not globally reliable. The algorithm tends to move toward the *local minimum or maximum* of the function to which the initial guess x_0 belongs. If the derivative changes sign or vanishes along the way, the tangent can point toward an entirely wrong region.

Consider $f(x) = x^3 - 2x + 2$ with initial guess $x_0 = 0$. Here, the tangent at x_0 leads to $x_1 = 1$, and the next tangent from x_1 brings us back to $x_2 = 0$. The iteration becomes trapped in a two-cycle and never approaches the true root.

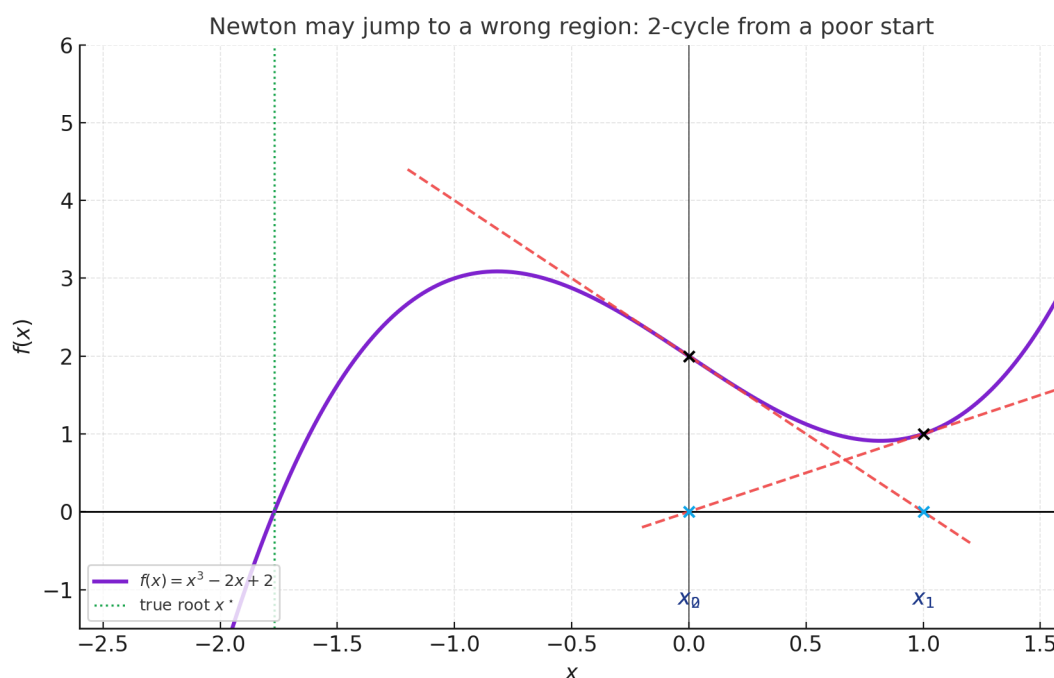


Figure: Newton with $f(x) = x^3 - 2x + 2$. With the poor initial guess $x_0 = 0$, the method falls into a two-cycle $0 \leftrightarrow 1$ and fails to reach the true root $x^* \approx -1.769$.

Note

Newton's method is locally powerful but not globally reliable. Good initial guesses are crucial; in practice, one often combines Newton's step with *bracketing* or *damping* to prevent wild jumps.

1.7 Multi-Point and Higher-Order Methods

So far, all iterative schemes have used *a single point* x_k to produce the next approximation. However, we can often improve performance by using information from *several previous iterates*. This leads to the family of **multi-point methods**.

Motivation Newton's method achieves fast (quadratic) convergence, but it requires evaluating $f'(x_k)$ at each step. In many real problems, especially when f is obtained from simulations or experiments, the derivative is either unavailable or too expensive to compute numerically.

The Secant Method The **Secant method** answers exactly that question. It replaces the true derivative $f'(x_k)$ in Newton's update with a finite difference computed from the last two iterates:

$$f'(x_k) \approx \frac{f(x_k) - f(x_{k-1})}{x_k - x_{k-1}}.$$

Substituting this into Newton's formula gives:

$$x_{k+1} = x_k - f(x_k) \frac{x_k - x_{k-1}}{f(x_k) - f(x_{k-1})}.$$

Why this formula makes sense. The key idea is geometric: if we connect the last two points $(x_{k-1}, f(x_{k-1}))$ and $(x_k, f(x_k))$ by a straight line, that line is our local approximation of the curve. The intersection of this secant with the x -axis provides the next approximation x_{k+1} .

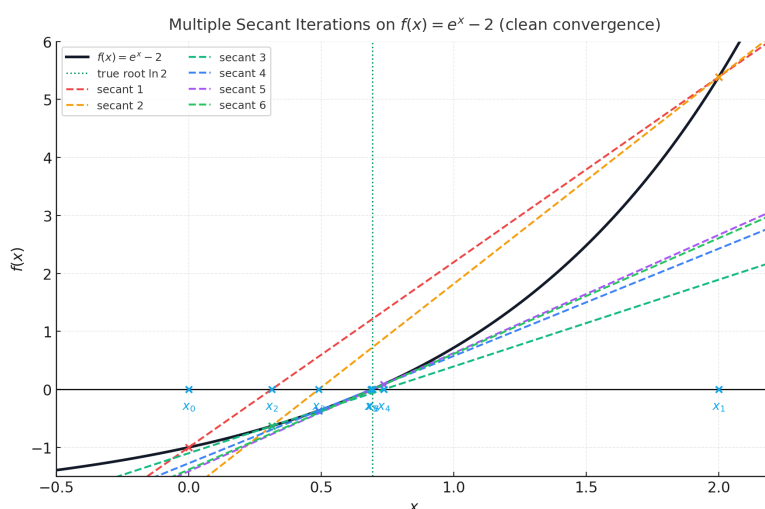


Figure: Successive Secant steps on $f(x) = e^x - 2$ with $x_0 = 0$, $x_1 = 2$. Each dashed line is a new secant using the two latest points; the intersections x_{k+1} approach the true root $\ln 2$ cleanly.

Mathematically, the secant through $(x_{k-1}, f(x_{k-1}))$ and $(x_k, f(x_k))$ is

$$y = f(x_k) + \frac{f(x_k) - f(x_{k-1})}{x_k - x_{k-1}} (x - x_k).$$

Setting $y = 0$ gives

$$x_{k+1} = x_k - f(x_k) \frac{x_k - x_{k-1}}{f(x_k) - f(x_{k-1})},$$

which is exactly the update formula.

This rule is *consistent*: if $x_k = x^*$ is a true root ($f(x^*) = 0$), then $x_{k+1} = x^*$.

Order of convergence. Let the true root be x^* , and define the local error $e_k = x_k - x^*$. Near the root, we can approximate the function by its Taylor expansion:

$$f(x) \approx f'(x^*)(x - x^*) + \frac{1}{2}f''(x^*)(x - x^*)^2.$$

Substituting this approximation into the secant update and simplifying the resulting algebra

(details omitted), one obtains that the next error behaves approximately as

$$e_{k+1} \approx C e_k e_{k-1},$$

where C is a constant depending on $f''(x^*)$ and $f'(x^*)$.

Now, assume the errors follow an asymptotic pattern $|e_{k+1}| \approx \lambda |e_k|^p$ for some order $p > 1$. Using the recursive form above and canceling constants gives

$$|e_{k+1}| \approx C |e_k| |e_{k-1}| \quad \Rightarrow \quad |e_k|^p \propto |e_k| |e_{k-1}| \quad \Rightarrow \quad p^2 = p + 1.$$

Solving this quadratic yields

$$p = \frac{1 + \sqrt{5}}{2} \approx 1.618,$$

the famous **golden ratio**. This means that the Secant method converges *superlinearly*: faster than linear schemes (like fixed-point iteration), but slightly slower than Newton's quadratic convergence.

2 Multidimensional Newton's Method

Up to now, our focus has been on finding the root of a scalar nonlinear equation:

$$f(x) = 0, \quad x \in \mathbb{R}.$$

We now extend this idea to *systems* of nonlinear equations, where we seek a vector

$$\mathbf{x}^* \in \mathbb{R}^n \quad \text{such that} \quad F(\mathbf{x}^*) = \mathbf{0}, \text{ with}$$

$$F(\mathbf{x}) = \begin{bmatrix} f_1(x_1, \dots, x_n) \\ \vdots \\ f_n(x_1, \dots, x_n) \end{bmatrix}.$$

2.1 Idea and Derivation

Just as Newton's method for a scalar function uses a local linear approx, the multidimensional version relies on a **first-order Taylor expansion** of F near the current iterate $\mathbf{x}^{(k)}$:

$$F(\mathbf{x}^{(k)} + \Delta \mathbf{x}) \approx F(\mathbf{x}^{(k)}) + J_F(\mathbf{x}^{(k)}) \Delta \mathbf{x},$$

where $J_F(\mathbf{x}^{(k)})$ is the *Jacobian matrix*

$$J_F(\mathbf{x}) = \begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \cdots & \frac{\partial f_1}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial f_n}{\partial x_1} & \cdots & \frac{\partial f_n}{\partial x_n} \end{bmatrix}.$$

Setting $F(\mathbf{x}^{(k)} + \Delta \mathbf{x}) = 0$ and neglecting higher-order terms gives the linear system

$$J_F(\mathbf{x}^{(k)}) \Delta \mathbf{x} = -F(\mathbf{x}^{(k)}),$$

whose solution $\Delta \mathbf{x}$ defines the update: $\boxed{\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} - J_F(\mathbf{x}^{(k)})^{-1} F(\mathbf{x}^{(k)})}$.

2.2 Implementation Viewpoint

In practice, the inverse of J_F is not computed explicitly. Instead, we *solve* the linear system

$$J_F(\mathbf{x}^{(k)}) \mathbf{s}^{(k)} = -F(\mathbf{x}^{(k)}),$$

for the step $\mathbf{s}^{(k)}$, and set

$$\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} + \mathbf{s}^{(k)}.$$

This formulation highlights the two essential computational ingredients:

- evaluating both $F(\mathbf{x})$ and its Jacobian $J_F(\mathbf{x})$,
- solving an $n \times n$ linear system at each iteration.

Note

When n is large, computing and inverting the Jacobian dominates the cost. This motivates the *quasi-Newton methods*, which build approximate Jacobians at lower cost.

2.3 Connection to Fixed-Point Form and Local Behavior

Newton's method can be viewed as a fixed-point iteration

$$\mathbf{x}_{k+1} = \Phi(\mathbf{x}_k), \quad \Phi(\mathbf{x}) = \mathbf{x} - J_F(\mathbf{x})^{-1}F(\mathbf{x}).$$

To study convergence, we examine the Jacobian of Φ :

$$J_\Phi(\mathbf{x}) = I - \frac{\partial}{\partial \mathbf{x}}(J_F(\mathbf{x})^{-1}F(\mathbf{x})).$$

Applying the product rule,

$$J_\Phi(\mathbf{x}) = I - [J_F(\mathbf{x})^{-1}J_F(\mathbf{x}) - J_F(\mathbf{x})^{-1}(\nabla J_F)(\mathbf{x})[F(\mathbf{x})]] = J_F(\mathbf{x})^{-1}(\nabla J_F)(\mathbf{x})[F(\mathbf{x})].$$

Explanation:

- The derivative of the identity map is I .
- The derivative of $J_F^{-1}F$ uses the product rule and the directional form of the identity $\nabla(J_F^{-1})[v] = -J_F^{-1}(\nabla J_F)[v]$, which is a simplified version of $\nabla(J_F^{-1}) = -J_F^{-1}(\nabla J_F)J_F^{-1}$.
- Simplifying the terms gives $I - [I - J_F^{-1}(\nabla J_F)[F]] = J_F^{-1}(\nabla J_F)[F]$.
- The term $(\nabla J_F)(\mathbf{x})$ represents the derivative of the Jacobian, which is a third-order tensor containing all second derivatives of F . In other words, it is **closely related to the Hessian**: for a scalar function $F : \mathbb{R}^n \rightarrow \mathbb{R}$, $(\nabla J_F)(\mathbf{x})$ is *exactly the Hessian matrix*; for a vector-valued $F : \mathbb{R}^n \rightarrow \mathbb{R}^m$, it collects the Hessians of all component functions F_i .
- Its action on $F(\mathbf{x})$ means contraction along one index:

$$[(\nabla J_F)(\mathbf{x})[F(\mathbf{x})]]_{ij} = \sum_k F_k(\mathbf{x}) \frac{\partial^2 F_i(\mathbf{x})}{\partial x_j \partial x_k}.$$

At the true root $\mathbf{x}^*$ we have $F(\mathbf{x}^*) = 0$, hence

$$J_\Phi(\mathbf{x}^*) = 0.$$

Because the derivative (Jacobian) of the iteration function vanishes at the fixed point, the method is *quadratically convergent*:

$$\|\mathbf{x}_{k+1} - \mathbf{x}^*\| \leq C \|\mathbf{x}_k - \mathbf{x}^*\|^2.$$

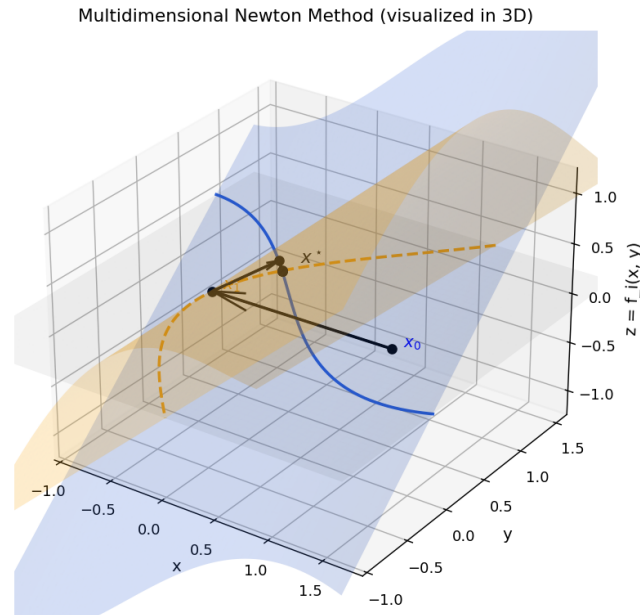


Figure 1: 3D visualization of the multidimensional Newton method. The two semi-transparent surfaces represent the nonlinear equations $z = f_1(x, y)$ and $z = f_2(x, y)$. Their intersections with the plane $z = 0$ correspond to the level sets $f_1(x, y) = 0$ and $f_2(x, y) = 0$. The Newton iterations $x_0 \rightarrow x_1 \rightarrow x^*$ are shown as arrows on the $z = 0$ plane, illustrating how the method converges to the common root of both functions.

Note

The **quadratic convergence** of the multidimensional Newton method holds only under specific regularity conditions. In particular, if the function $F : \mathbb{R}^n \rightarrow \mathbb{R}^n$ is twice continuously differentiable ($F \in C^2$) in a neighborhood of the true root $\mathbf{x}^*$, and the Jacobian $J_F(\mathbf{x}^*)$ is invertible, then there exists a neighborhood $\mathcal{U}$ of $\mathbf{x}^*$ such that, for any initial guess $\mathbf{x}^{(0)} \in \mathcal{U}$, the sequence

$$\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} - J_F(\mathbf{x}^{(k)})^{-1} F(\mathbf{x}^{(k)})$$

converges to $\mathbf{x}^*$ with *quadratic order*.

If $J_F(\mathbf{x}^*)$ is singular or nearly singular, these guarantees break down and the iteration may stagnate, slow to linear convergence, or diverge entirely.

3 Damped/Line Search Newton Method

Newton's method converges quadratically when started sufficiently close to the true root. However, if the initial guess is far from $\mathbf{x}^*$, the tangent step can easily overshoot or even diverge. Intuitively, Newton assumes the function is well approximated by its tangent in the current neighborhood, an assumption that breaks down if we start too far away. To improve robustness, we introduce a damping factor that *controls the step size* along the Newton direction. This is the idea behind the **Line Search Newton Method**.

3.1 Motivation (1D Intuition)

Consider a scalar nonlinear function $f : \mathbb{R} \rightarrow \mathbb{R}$. Standard Newton takes the full tangent step:

$$x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)}.$$

If $f'(x_k)$ is small or the function is highly curved, this step may shoot far across the root, sending x_{k+1} to a completely different region. Each iteration can then amplify the error instead of reducing it.

Idea: we introduce a *damping parameter* $\alpha_k \in (0, 1]$ that scales the step length:

$$x_{k+1} = x_k - \alpha_k \frac{f(x_k)}{f'(x_k)}.$$

Note

From a geometric viewpoint, inserting the factor $\alpha_k \in (0, 1]$ is equivalent to scaling the tangent slope. If we imagine moving α_k to the denominator,

$$x_{k+1} = x_k - \frac{f(x_k)}{(1/\alpha_k) f'(x_k)},$$

we see that the derivative effectively becomes *larger in magnitude*. A steeper tangent intersects the x -axis sooner, this shortens the Newton step and prevents overshooting the root.

The parameter α_k is chosen such that the new point x_{k+1} still moves in the Newton direction, but only part of the way, ensuring a consistent decrease in $|f(x)|$ (or in its squared norm, $f(x)^2$).

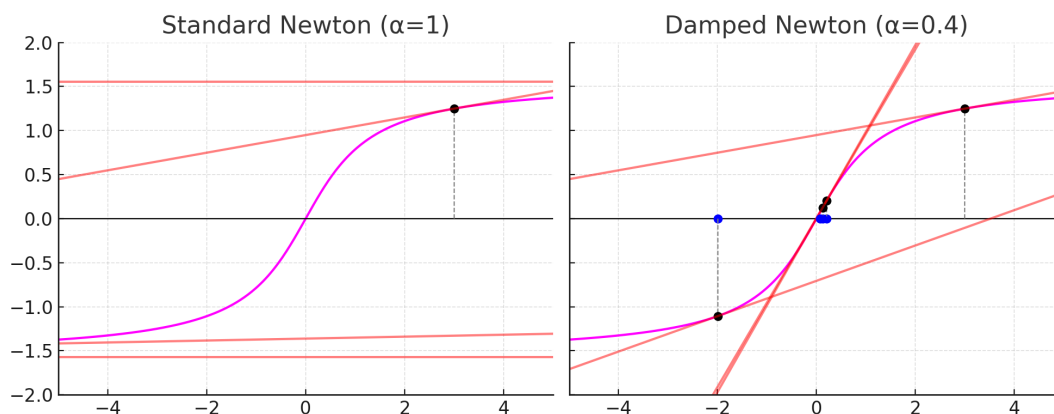


Figure: Comparison of standard Newton (left) and damped Newton (right) for $f(x) = \arctan(x)$. The damping factor $\alpha_k < 1$ prevents overshooting and ensures monotone convergence.

Note

The damping factor α_k controls how aggressively Newton moves along the tangent direction. When $\alpha_k = 1$, we recover the classical (full) Newton step. When $\alpha_k < 1$, the iteration becomes more cautious: progress per step is slower, but the sequence $\{x_k\}$ is far less likely to diverge. In practice, a well-chosen α_k can turn an unstable Newton iteration into one that converges globally.

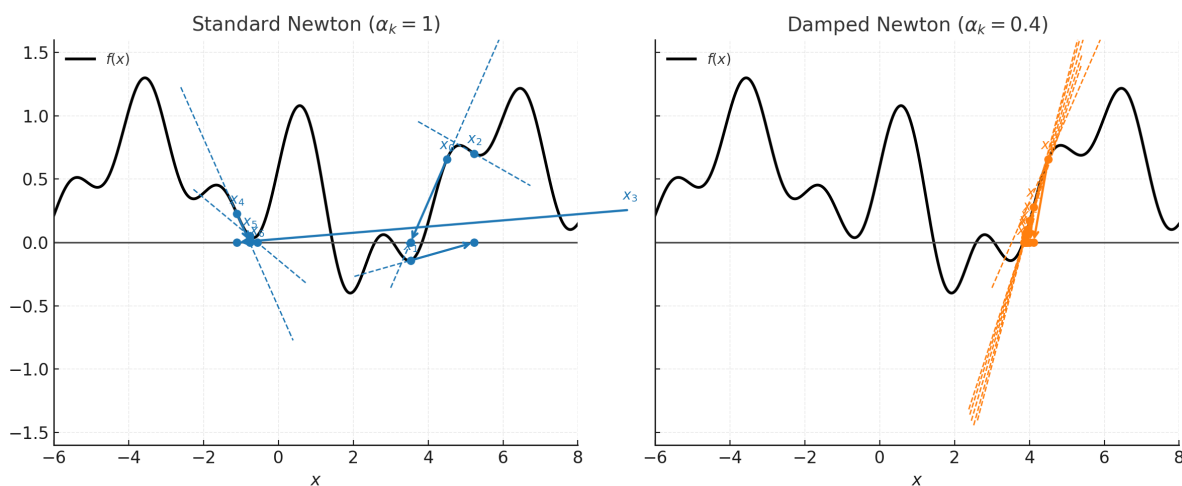


Figure: Standard Newton (left) vs. damped Newton (right) on a multimodal function with a single valley crossing $y = 0$. Dashed lines are tangents at x_k ; their intersections with the x -axis are the next iterates x_{k+1} . Damping ($\alpha_k < 1$) shortens the tangent step and keeps the iterates in the correct basin.

3.2 (Optional) Choosing the Damping Parameter

In practice, the damping factor α_k controls how far we move along the Newton direction. A common goal is to make sure each step *reduces* the size of the residual $|f(x)|$ rather than increasing it.

A simple and popular choice is the **backtracking rule**:

- Start with $\alpha_k = 1$ (the full Newton step).
- If this step makes $|f(x_{k+1})|$ larger, reduce α_k (for example, halve it).
- Repeat until the residual decreases: $|f(x_{k+1})| < |f(x_k)|$.

This ensures that each iteration makes measurable progress toward the root, even if we start far away or the function is highly nonlinear.

Note

This heuristic guarantees that Newton's method will not diverge just because of a poor starting point or an overly large tangent step. While smaller α_k slows convergence near the root, it greatly improves *global robustness*, the method succeeds in more cases.

3.3 Summary

The damped Newton method blends the speed of Newton's quadratic convergence with the robustness of controlled step sizes.

Solve: $J_F(\mathbf{x}_k) \mathbf{p}_k = -F(\mathbf{x}_k)$,
Choose: $\alpha_k \in (0, 1]$ by line search,
Update: $\mathbf{x}_{k+1} = \mathbf{x}_k + \alpha_k \mathbf{p}_k$.

When the iterates are near $\mathbf{x}^*$, $\alpha_k \rightarrow 1$ and quadratic convergence is recovered; when they are far, the damping prevents divergence.

4 Quasi–Newton (Broyden) Method

Newton’s method relies on the exact Jacobian $DF(\mathbf{x}_k)$, which must be computed and inverted at every iteration. This becomes impractical when:

- the Jacobian is expensive or impossible to compute analytically,
- $DF(\mathbf{x}_k)$ is *nearly singular* or ill-conditioned,
- the function F is not differentiable everywhere,
- or we are solving large systems where forming $DF(\mathbf{x})$ explicitly is too costly.

In such cases, we can replace the true Jacobian with an *approximation* that is updated iteratively using information from previous steps. This leads to the family of **Quasi–Newton methods**.

Note

Just as the Secant method avoids computing derivatives in one dimension, the Quasi–Newton approach extends this idea to higher dimensions by approximating the Jacobian from differences of $F(\mathbf{x})$ values.

4.1 From Secant to Broyden’s Update

Recall that in 1D, the **Secant method** replaces the derivative $f'(x_k)$ with a finite difference:

$$f'(x_k) \approx \frac{f(x_k) - f(x_{k-1})}{x_k - x_{k-1}}.$$

In multiple dimensions, the Jacobian $J_k \approx DF(\mathbf{x}_k)$ must satisfy the corresponding *secant condition*:

$$J_k (\mathbf{x}_k - \mathbf{x}_{k-1}) = F(\mathbf{x}_k) - F(\mathbf{x}_{k-1}).$$

However, this single vector equation does not uniquely determine J_k : there are infinitely many matrices that satisfy it. In higher dimensions we cannot simply “divide” by $(\mathbf{x}_k - \mathbf{x}_{k-1})$ as in the scalar case.

Broyden’s idea. We look for a new matrix J_k that changes the previous Jacobian J_{k-1} as little as possible, while still satisfying the new secant condition. Intuitively, we want to preserve all information that is still valid from the previous step and only correct what is necessary.

$$\min_{J_k} \|J_k - J_{k-1}\|_F \quad \text{subject to} \quad J_k s_k = y_k,$$

where

$$s_k = \mathbf{x}_k - \mathbf{x}_{k-1}, \quad y_k = F(\mathbf{x}_k) - F(\mathbf{x}_{k-1}).$$

Broyden update:

$$J_k = J_{k-1} + \frac{(y_k - J_{k-1}s_k) s_k^T}{\|s_k\|_2^2}.$$

Optional: How the formula arises (intuitive view). We imagine adding a small correction ΔJ to the previous Jacobian:

$$J_k = J_{k-1} + \Delta J.$$

This correction must ensure that the new Jacobian satisfies the updated secant condition:

$$(J_{k-1} + \Delta J)s_k = y_k \quad \Rightarrow \quad \Delta J s_k = y_k - J_{k-1}s_k.$$

The right-hand side is the “error” in the prediction of the change y_k , based on the old Jacobian. There are infinitely many matrices ΔJ that could fix this, so we choose the smallest possible change, one that only acts in the direction of s_k :

$$\Delta J = (y_k - J_{k-1}s_k) v^T.$$

We then determine v such that $\Delta J s_k = y_k - J_{k-1}s_k$. Substituting gives:

$$(y_k - J_{k-1}s_k)(v^T s_k) = y_k - J_{k-1}s_k \quad \Rightarrow \quad v = \frac{s_k}{\|s_k\|_2^2}.$$

Finally, substituting v back yields the celebrated **Broyden update**:

$$J_k = J_{k-1} + \frac{(y_k - J_{k-1}s_k) s_k^T}{\|s_k\|_2^2}.$$

Note

The correction term

$$\frac{(y_k - J_{k-1}s_k) s_k^T}{\|s_k\|_2^2}$$

is a *rank-one update*: it modifies J_{k-1} only along the direction of s_k , leaving all orthogonal directions unchanged. This ensures the smallest possible change (in Frobenius norm) that makes the new Jacobian consistent with the most recent step.

Optional intuition. In one dimension, the Broyden update reduces exactly to the Secant slope:

$$J_k = \frac{f(x_k) - f(x_{k-1})}{x_k - x_{k-1}}.$$

Hence, Broyden’s method can be viewed as the natural multidimensional generalization of the Secant method, derivative-free and still rapidly convergent.

Note

When J_k is invertible, we can directly update its inverse using the **Sherman–Morrison formula**, avoiding explicit inversion at each step.

This formula provides an efficient way to update $(J_k)^{-1}$ after a rank-one change to J_k , without recomputing the full inverse. It is derived from the identity:

$$(A + uv^T)^{-1} = A^{-1} - \frac{A^{-1}uv^T A^{-1}}{1 + v^T A^{-1}u},$$

valid whenever $1 + v^T A^{-1}u \neq 0$.

In the context of Broyden's method, $A = J_{k-1}$ and u, v represent the correction terms from the secant condition. Using this update keeps the algorithm efficient: instead of inverting a new Jacobian at each step, we just perform a low-cost vector–matrix update.

Exam-Type Exercise:

Problem Statement

Consider the nonlinear system:

$$\begin{cases} x_1^2 + x_2^2 - 4 = 0, \\ e^{x_1} + x_2 - 1 = 0. \end{cases}$$

Use **Newton's Method** to approximate the solution $\mathbf{x}^* = (x_1^*, x_2^*)^T$ of this system, starting from $\mathbf{x}^{(0)} = (1, 1)^T$.

- (a) Write the nonlinear function $F(\mathbf{x})$ and its Jacobian $J_F(\mathbf{x})$.
- (b) Apply the Newton iteration

$$\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} - [J_F(\mathbf{x}^{(k)})]^{-1} F(\mathbf{x}^{(k)}),$$

until $\|F(\mathbf{x}^{(k)})\|_2 < 10^{-3}$.

- (c) Report the approximate solution $\mathbf{x}_{\text{num}}^*$ and the number of iterations required.

Note

If the full Newton step diverges, use a *damped step*:

$$\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} - \alpha_k [J_F(\mathbf{x}^{(k)})]^{-1} F(\mathbf{x}^{(k)}), \quad \alpha_k \in (0, 1].$$

Solution Outline and Discussion

We define:

$$F(\mathbf{x}) = \begin{bmatrix} x_1^2 + x_2^2 - 4 \\ e^{x_1} + x_2 - 1 \end{bmatrix}, \quad J_F(\mathbf{x}) = \begin{bmatrix} 2x_1 & 2x_2 \\ e^{x_1} & 1 \end{bmatrix}.$$

Step 1: Compute the Newton direction. At each iteration, we solve the linear system

$$J_F(\mathbf{x}_k) \mathbf{p}_k = F(\mathbf{x}_k), \quad \text{then set } \mathbf{x}_{k+1} = \mathbf{x}_k - \mathbf{p}_k.$$

This corresponds to intersecting the tangent plane of the nonlinear map with the zero plane $F = 0$, the multidimensional analogue of the 1D tangent step.

Step 2: Perform successive iterations. Starting with $\mathbf{x}^{(0)} = (1, 1)^T$, we evaluate:

$$F(\mathbf{x}^{(0)}) = \begin{bmatrix} 1^2 + 1^2 - 4 \\ e^1 + 1 - 1 \end{bmatrix} = \begin{bmatrix} -2 \\ e \end{bmatrix},$$

$$J_F(\mathbf{x}^{(0)}) = \begin{bmatrix} 2 & 2 \\ e & 1 \end{bmatrix}.$$

Solving $J_F(\mathbf{x}^{(0)}) \mathbf{p}^{(0)} = F(\mathbf{x}^{(0)})$ gives

$$\mathbf{p}^{(0)} = \begin{bmatrix} \frac{1+e}{e-1} \\ -\frac{e(2+e)}{e-1} \end{bmatrix} \approx \begin{bmatrix} 2.164 \\ -3.164 \end{bmatrix}.$$

Hence

$$\mathbf{x}^{(1)} = (1, 1)^T - \mathbf{p}^{(0)} \approx (-1.164, 4.164)^T.$$

Repeating the iteration (e.g. using a numerical solver) yields convergence to one of the two real solutions of the system:

$$\boxed{\mathbf{x}_1^* \approx (-1.8163, 0.8374)^T, \quad \mathbf{x}_2^* \approx (1.0042, -1.7296)^T.}$$

Step 3: Convergence remarks. The Jacobian remains nonsingular near the solutions:

$$\det J_F(\mathbf{x}) = 2x_1 - 2x_2 e^{x_1} \neq 0,$$

so quadratic convergence of Newton's method is expected locally. However, depending on the initial guess, the method may converge to either of the two solutions or diverge if started too far away.